Carelink - Non-Profit Quiz/Tutor Bot

Institution Name: Medicaps University – Datagami Skill Based Course

Sr no	Student Name	Enrolment Number
1	Neha Tamboli	EN22CS301641
2	Riddhima Taose	EN22CS301642
3	Uday Patidar	EN23CS3L1023
4	Rohan Rawat	EN23CS3L1016
5	Naman Dwivedi	EN22CS301627

Group Name: 04D6 Project

Number: DO-15

Industry Mentor Name:

University Mentor Name: Ajaj Khan

Academic Year: 2025-26

Table of Contents

1. **Introduction.**
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.

2. **System Design.**
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design 2.5. Key Design Considerations
   2.6. API Catalogue.

3. **Key Design Considerations**
   3.1 Infrastructure Catalogue & Data Design
   3.2 Data Model Design

4. **References**

1. Introduction

1.1. Scope of the document

This Low-Level Design (LLD) document provides the technical details and implementation approach for the **AI-Based Non-Profit Quiz/Tutor Bot System**. The system is designed to generate quiz questions from donor emails, evaluate user responses using a Large Language Model (LLM), and provide meaningful feedback to improve user understanding.

The document covers system architecture, component interactions, data flow, API design, and deployment strategies using Python-based backend technologies, vector databases, and AI models. It also includes chatbot functionality for interactive and personalized learning.

The scope of this document includes:

- **System architecture and component design**
  Defines the structure of frontend, backend, quiz module, chatbot, and AI integration.

- **Data handling and quiz generation**
  Explains how donor emails are processed into questions and how user responses are managed.

- **API design and endpoints**
  Includes APIs for question generation, answer evaluation, and chatbot interaction.

- **Python backend using FastAPI**
  Handles all logic, processing, and communication between components.

- **Vector database integration (ChromaDB/FAISS)**
  Enables semantic search and context-based question generation.

- **LLM integration**
  Used for answer evaluation, scoring, and explanation generation.

- **Quiz and learning workflow**
  Supports adaptive learning through feedback and repeated questioning.

- **Chatbot module (RAG-based)**
  Provides conversational responses using vector data and LLM.

- **Deployment and DevOps support**
  Includes Docker, Kubernetes, and CI/CD pipeline setup.

- **Security, monitoring, and logging**
  Ensures system reliability, safety, and performance tracking.

This document acts as a technical guide for developers building the AI-based Quiz/Tutor Bot and provides deployment guidelines for Gen-AI engineers responsible for system infrastructure.

1.2. Intended Audience

This document is intended for stakeholders involved in the design, development, and deployment of the **AI-Based Non-Profit Quiz/Tutor Bot System**. It provides detailed technical insights to support implementation, integration, and system management **Backend Developers**: For implementing Python-based AI agents, Lang Chain workflows, and API endpoints

- **Backend Developers**
  For implementing the Python-based backend using FastAPI, quiz logic, API endpoints, and system integration.
- **AI/ML Engineers**
  For understanding LLM integration, prompt engineering, and answer evaluation mechanisms.
- **DevOps Engineers**
  For managing containerization (Docker), Kubernetes deployment, and CI/CD pipeline configuration.
- **Database Administrators**
  For handling vector database setup, optimization, and data storage for donor emails and embeddings.
- **Frontend Developers**
  For developing the user interface and integrating it with backend APIs for quiz and chatbot interaction.
- **QA Engineers**
  For testing system functionality, API performance, and AI response accuracy.
- **Security Engineers**
  For implementing authentication, securing API keys, and ensuring data protection.
- **Project Mentors and Supervisors**
  For reviewing system design, validating architecture, and evaluating project implementation.
- **System Architects**
  For analyzing design patterns, scalability, and overall system structure.

1.3. System Overview

The **AI-Based Non-Profit Quiz/Tutor Bot System** is an intelligent learning platform designed to enhance understanding of non-profit communication through interactive quizzes and chatbot-based guidance. The system utilizes donor emails as a knowledge source to generate meaningful questions, evaluate user responses using Large Language Models (LLMs), and provide contextual feedback to improve learning outcomes.

The platform integrates vector databases and AI models to create a dynamic and personalized learning experience. It not only assesses user knowledge but also identifies gaps and reinforces concepts through explanation-based feedback and continuous interaction.

Core Functionalities:

1. **Quiz Generation**
   Automatically generates questions from donor emails using semantic retrieval techniques, ensuring relevant and context-based assessments.
2. **Answer Evaluation**
   Evaluates user responses using LLMs by analyzing meaning rather than exact matches and assigns scores accordingly.
3. **Contextual Explanation**
   Provides detailed explanations for correct and incorrect answers, helping users understand concepts clearly.
4. **Personalized Learning Loop**
   Tracks user performance, identifies weak areas, and adapts future questions to improve learning outcomes.
5. **Chatbot Interaction**
   Allows users to ask queries and receive intelligent, context-aware responses using Retrieval-Augmented Generation (RAG).
6. **Semantic Search (Vector DB)**
   Retrieves relevant donor email content using embeddings stored in vector databases like ChromaDB or FAISS.
7. **Progress Tracking and Feedback**
   Monitors user performance, quiz scores, and learning progress to provide continuous improvement insights.

Technology Architecture:

The system follows a modern AI-driven and DevOps-oriented architecture designed to support scalable, interactive, and intelligent learning. It leverages Python-based backend services, vector databases, and Large Language Models (LLMs) to deliver quiz generation, answer evaluation, and chatbot functionality. The architecture ensures modularity, efficient data processing, and seamless deployment using containerization and orchestration tools.

- **Frontend Layer**
  Provides a user interface for quiz interaction, answer submission, chatbot access, and progress display.
- **Backend Layer**
  Python-based FastAPI server that manages quiz logic, API requests, and system workflows.
- **AI Processing Layer**
  Uses LLMs to evaluate answers, generate explanations, and support chatbot responses.
- **Vector Database Layer**
  Stores donor email embeddings and enables semantic search for question generation and context retrieval.
- **Containerization & Deployment**
  Uses Docker and Kubernetes for scalable deployment and environment consistency.
- **CI/CD Pipeline**
  Automates build, testing, and deployment using tools like GitHub Actions or Jenkins.

2. System Design

2.1. Application Design

The **AI-Based Non-Profit Quiz/Tutor Bot System** follows a modular and layered architecture that ensures clear separation of concerns and efficient interaction between components. The design supports quiz generation, answer evaluation, and chatbot-based learning using AI technologies

Architecture Pattern:

The application follows a multi-layered architecture with an integrated AI processing layer:

1. **Presentation Layer**
   Provides a user interface for quiz interaction, answer submission, and chatbot communication.

2. **API Layer**
   Handles RESTful API requests for fetching questions, submitting answers, and chatbot queries.

3. **Business Logic Layer**
   Manages quiz generation, answer evaluation, learning flow, and chatbot processing using LLMs.

4. **Data Access Layer**
   Handles vector database operations, donor email storage, semantic search, and data retrieval.

2.2. Process Flow

End-to-End Quiz and Learning Flow

The system follows a structured process to deliver an interactive quiz and chatbot-based learning experience. It starts with user interaction and ends with AI-based evaluation and feedback generation, creating a continuous learning loop.

Process Steps:

1. **User Interaction**
- User clicks the chatbot/quiz icon
- Chat interface opens with a welcome message
- User selects a donor email or starts the quiz

2. **Question Generation**
- System retrieves donor email from vector database
- Relevant content is selected using semantic search
- Quiz question is generated based on the selected email

3. **Answer Submission**
- User enters the answer in the chat input
- Frontend sends the response to backend API (/answer)
- Input is validated before processing

4. **AI Evaluation**
- LLM processes the question and user answer
- Evaluates based on meaning (not exact match)
- Generates:
  - Score
  - Correct explanation
  - Missing points

5. **Context Retrieval (Vector DB)**
- Relevant donor email context is fetched
- Helps improve evaluation and explanation quality
- Ensures accurate and context-aware feedback

6. **Feedback Generation**
- System returns evaluation results to frontend
- User sees score and detailed explanation
- Learning gaps are highlighted

7. **Personalized Learning**
- System tracks user performance
- Weak areas are identified
- Similar or adaptive questions are generated

8. **Chatbot Interaction (Optional)**
- User can ask questions anytime
- System retrieves relevant data from vector DB
- LLM generates contextual responses

Docker Container Lifecycle Flow:

The system follows a Docker-based containerized deployment approach to ensure scalability, portability, and smooth CI/CD integration for the AI Quiz/Tutor Bot application.

1. Development Phase

- Developers build and test the Python FastAPI backend and frontend locally
- Docker Compose is used to define and run multiple services (backend, frontend, vector DB)
- Code changes are reflected using hot-reload for fast development

2. Build Phase

- Docker file builds the application in two stages:
  - Build stage: installs dependencies and prepares application
  - Runtime stage: creates lightweight production image
- Final image is tagged with version for tracking

3. CI/CD Pipeline

- Code pushed to GitHub triggers automated pipeline
- Tests (unit + integration) are executed
- Docker image is built and pushed to container registry
- Deployment configuration is updated with new image

4. Deployment Phase (Kubernetes)

- Kubernetes deploys updated containers using new image
- Rolling updates ensure zero downtime
- Health checks verify service stability
- Old containers are removed after successful deployment

5. Runtime Operation

- Kubernetes manages container lifecycle automatically
- Auto-scaling adjusts based on system load
- Load balancer distributes traffic across instances
- Persistent storage ensures data consistency

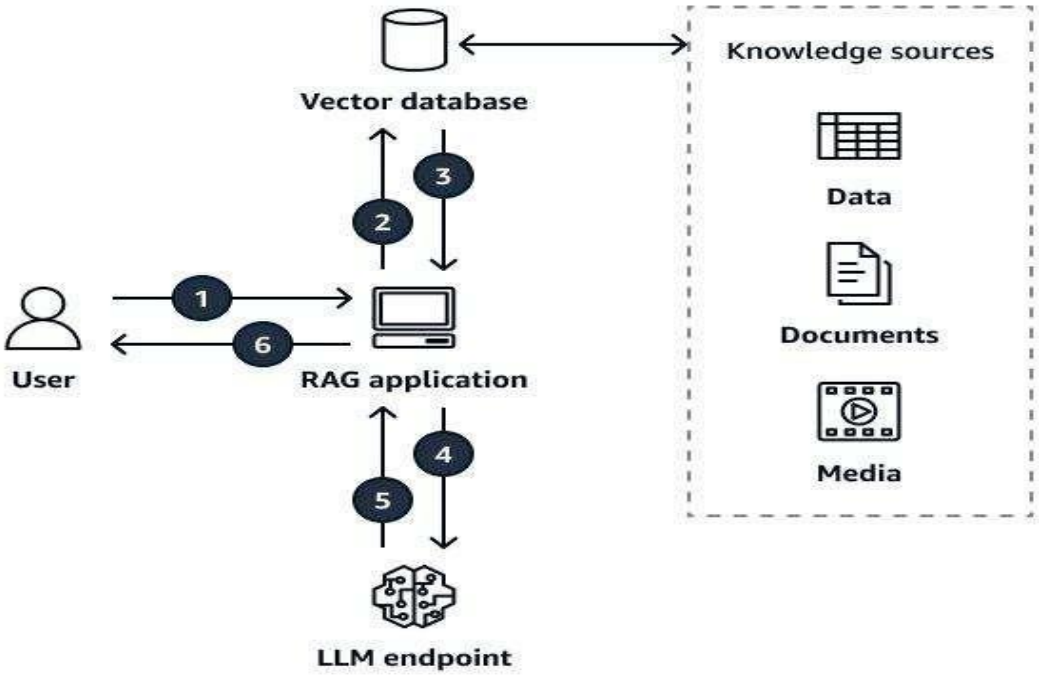
6. Monitoring & Logging

- Logs collected from backend and chatbot services
- Performance metrics monitored using tools like Prometheus
- Alerts triggered for errors or system failures
- Health endpoints continuously monitored

2.3.Information Flow

Data Flow Architecture:

The system implements a unidirectional data flow pattern with clear data transformation stages:



Data Flow Architecture

Information Type	Source	Destination
Donor Email Content	User / Database	Vector DB + Quiz Engine
Quiz Questions	Quiz Generator	Frontend UI
User Answers	Frontend	LLM Evaluation Module
Evaluation Results	LLM API	Database + Frontend
Context Embeddings	Embedding Model	Vector Database (ChromaDB)
Chatbot Queries	User Input	LLM + Vector DB
Feedback Explanations	LLM API	Frontend UI
User Progress Data	System Tracking	Database
Analytics Data	Database	Dashboard
System Logs	Backend Services	Logging System
Performance Metrics	Application	Monitoring System

2.4.Components Design

Core Application Components: The AI-Based Non-Profit Quiz/Tutor Bot System is built using a modular component-based architecture. Each component is responsible for a specific function such as quiz generation, answer evaluation, chatbot interaction, and data management. This separation ensures scalability, maintainability, and easy integration of AI services.

Core Application Components:

1. **Chatbot/Quiz Frontend Interface**
   Provides a real-time interactive interface for users to:
   - Start quizzes
   - Submit answers
   - Interact with chatbot
   - View feedback and scores
2. **Fast API Backend**
   Acts as the central processing layer that:
   - Handles API requests (/question, /answer, /chat)
   - Manages quiz flow and session logic
   - Connects frontend with AI and database layers
3. **LLM Orchestrator Module**
   Manages interaction with Large Language Models to:
   - Evaluate user answers
   - Generate explanations and scores
   - Power chatbot responses
   - Maintain conversational context
4. **Vector Database (ChromaDB / FAISS)**
   Used for:
   - Storing donor email embeddings
   - Performing semantic search

- Retrieving context for quiz questions and chatbot responses

5. **Database (PostgreSQL / MongoDB)**
   Stores:
   - User profiles
   - Quiz history
   - Scores and progress tracking
   - System logs

Infrastructure Components:

1. **Docker Containers**
   Provides isolated environments for:
   - Backend services
   - Frontend interface
   - Vector database

- **Kubernetes Cluster**
  Ensures:
  - Auto-scaling of services
  - High availability
  - Load balancing across services

3. **Ingress Controller**
   Handles:
   - Routing of external requests
   - API traffic management
   - Secure access to services

4. **Persistent Storage (Volumes)**
   Ensures:
   - Data durability for database and vector store
   - Data persistence across restarts

2.5.Key Design Considerations

Security Architecture : The AI-Based Non-Profit Quiz/Tutor Bot System is designed with strong emphasis on security, scalability, and reliable AI interaction. The architecture ensures safe handling of user data, secure API communication, and protected integration with LLM and vector databases.

Security Architecture

- **Authentication**
  The system uses JWT-based authentication to manage user access. Role-based access control is implemented for different user types such as admin, student, and system operator.
- **DataProtection**
  Sensitive data such as user responses, quiz history, and donor email content is protected using encryption during storage and transmission.
- **API Security**
  All API endpoints are secured using:
  - Input validation
  - Rate limiting to prevent abuse
  - Protection against injection attacks
- **Container Security**
  Docker containers run with restricted permissions using:
  - Non-root users
  - Minimal base images
  - Regular vulnerability scanning
- **SecretsManagement**
  API keys and sensitive credentials are securely managed using environment variables or Kubernetes secrets, ensuring no hardcoded sensitive data in the codebase.

3. Key Design Considerations (Scalability, Performance & Reliability)

The AI-Based Non-Profit Quiz/Tutor Bot System is designed to handle large-scale user interactions efficiently while maintaining high performance, reliability, and smooth AI-driven operations. The system ensures that quiz generation, answer evaluation, and chatbot responses remain fast and stable under varying loads.

Scalability Strategy

- **Horizontal Scaling**
  The system supports multiple backend instances to handle increased user traffic, with auto-scaling based on CPU and memory usage.

- **Independent Service Scaling**
  Quiz engine, chatbot module, and LLM processing can scale independently based on demand.

- **Caching Layer**
  Redis is used to store frequently accessed quiz data, chatbot responses, and user session states for faster performance.

- **Database Optimization**
  Efficient indexing and schema design ensure smooth handling of user data, quiz history, and embeddings.

Performance Optimization

- **Optimized Container Builds**
  Docker multi-stage builds reduce image size and improve deployment speed.

- **Connection Pooling**
  Efficient reuse of database and API connections reduces latency.

- **Asynchronous Processing**
  Non-blocking execution is used for LLM calls and vector database queries to improve response time.

- **Response Caching**
  Frequently asked questions and embeddings are cached to reduce repeated computations.

Reliability Engineering

- **Health Checks**
  Continuous monitoring of backend, chatbot, and database services ensures system availability.

- **Failure Handling**
  If LLM services fail, fallback logic provides basic rule-based responses.

- **Graceful Degradation**
  System continues to function with limited features during high load or partial failures.

- **Data Backup**
  Regular backups ensure that user progress, quiz history, and system data are never lost.

3.1. Infrastructure Catalogue & Data Design

Infrastructure Endpoints

The system exposes RESTful APIs to manage quiz generation, answer evaluation, chatbot interaction, and system monitoring. These endpoints enable communication between frontend, backend, vector database, and LLM services.

1. **Quiz Question API**
   POST /api/v1/question
   Generates quiz questions from donor email data using vector search.

2. **Answer Evaluation API**
   POST /api/v1/answer
   Submits user answers and returns LLM-based evaluation including score and explanation.

3. **Chatbot API**
   POST /api/v1/chat
   Handles user queries and generates AI-powered responses using RAG (Retrieval-Augmented Generation).

4. **Progress & Dashboard API**
   GET /api/v1/dashboard
   Provides user progress, quiz history, and performance analytics.

5. **Session Management API**
   GET /api/v1/session
   Tracks user quiz sessions, current question state, and learning flow.

3.2 Data Model Design

Primary Entities:

- **Donor Emails Dataset**
  Stores raw donor communication used for generating quiz content.
- **Quiz Questions**
  Generated questions based on semantic understanding of donor emails.
- **User Responses**
  Stores answers submitted by users for evaluation.
- **Evaluation Results**
  Contains scores, feedback, and LLM-generated explanations.
- **Chatbot Interactions**
  Stores conversation history between user and AI tutor.
- **User Progress Data**
  Tracks performance, weak areas, and learning improvements.
-

Relationships

- **Email → Question (1:N)**
  One donor email can generate multiple quiz questions.
- **Question → Response (1:N)**
  Multiple user answers can be linked to a single question.
- **Response → Evaluation (1:1)**
  Each answer receives one AI-based evaluation.
- **User → Progress (1:N)**
  Each user has multiple quiz attempts and progress records.

Vector Storage Design

- **ChromaDB / FAISS** is used to store embeddings of donor emails
- Enables **semantic search for quiz generation and chatbot context retrieval**

- Works alongside relational database for structured data storage

4. References

- FastAPI Documentation
- OpenAI / LLM API Documentation
- ChromaDB / FAISS Vector Database Docs
- Kubernetes Official Documentation
- DevOps Best Practices